ER Visual Designer — System Design

A web-based tool for visually modelling relational database schemas for a selected database engine, validating them against that engine's rules, and generating ready-to-run DDL.

Document System Design Specification **Version** 1.1 (draft for review) **Date** 8 November 2026

Status Design — no implementation

Contents

1. Purpose and scope
   - 1.1 Goals
   - 1.2 Non-goals (initial release)
   - 1.3 Terminology
2. Requirements
   - 2.1 Functional requirements
   - 2.2 Non-functional requirements
3. Architecture overview
   - 3.1 Component diagram
   - 3.2 Component responsibilities
   - 3.3 Technology choices
   - 3.4 Package structure
4. Domain model
   - 4.1 Class diagram
   - 4.2 Key modelling decisions
   - 4.3 Serialised project format (`.erd.json`)
5. Database engine dialects
   - 5.1 Dialect plug-in interface
   - 5.2 Initial engine matrix
   - 5.3 Logical type mapping
6. Validation engine
   - 6.1 Design
   - 6.2 Rule catalogue (initial)
   - 6.3 Issue presentation
7. State management, commands and undo/redo
8. DDL generation
   - 8.1 Pipeline
   - 8.2 Example output (same model, two engines)
9. User interface design
   - 9.1 Chosen visual direction — Sunrise Studio
   - 9.2 Workspace layout
   - 9.3 Key interactions
   - 9.4 Design tokens
   - 9.5 Component styling
   - 9.6 Accessibility of the palette
   - 9.7 Responsive behaviour
10. UI interaction sequences
    - 10.1 Create a table
    - 10.2 Add and edit a column
    - 10.3 Drag a table on the canvas
    - 10.4 Draw a foreign-key relationship
    - 10.5 Select an engine and revalidate
    - 10.6 Navigate to a problem and apply a quick fix
    - 10.7 View and export DDL
    - 10.8 Connect to a database and reverse-engineer (Phase 6)
    - 10.9 Excel import to DML (Phase 7)
    - 10.10 Live performance metrics (Phase 8)
11. Backend and API (Phase 6+)
    - 11.1 Deployment view
    - 11.2 API surface (indicative)
    - 11.3 Security
12. Quality and testing strategy
13. Risks and open questions

1. Purpose and scope

ER Visual Designer lets a user build a database schema on an interactive canvas instead of writing DDL by hand. The user picks a target engine (PostgreSQL, MySQL, SQL Server, Oracle or SQLite), then creates tables and columns, sets keys and constraints, and links columns to form relationships. The tool continuously checks the model against the engine's rules, explains every problem with a suggested fix, and produces engine-specific DDL.

This document describes the target architecture for the full product. Delivery is phased; the phase plan and per-phase designs are in the companion **ROADMAP** document.

Change log — v1.1 (8 Nov 2026): visual direction decided (**Sunrise Studio**, §9.1), with design tokens, component styling, contrast check and responsive rules (§9.4–9.7). v1.0 (6 Nov 2026): first draft.

1.1 Goals

#	Goal	Measure of success
G1	Model tables, columns, data types, PKs, FKs, nullability and defaults visually	Every column property is editable without writing SQL
G2	Interactive canvas for arranging tables and drawing relationships	Drag tables, pan/zoom, drag column-to-column to create an FK
G3	Engine-aware validation	Data types, naming rules and constraint rules checked per engine
G4	Actionable feedback	Every issue has a location, a plain-language explanation and, where possible, a one-click fix
G5	Engine-specific DDL	DDL that runs unmodified on the selected engine, in dependency-safe order
G6	Extensible engine support	A new engine is added as a plug-in, not by changing core code

1.2 Non-goals (initial release)

- Not a full SQL IDE: no arbitrary query editor or execution until the database-connectivity phases.
- No views, stored procedures, triggers or partitions in the first model version (the model is designed so they can be added).
- No real-time multi-user co-editing in the first release; the architecture leaves room for it (command log, see §7).

1.3 Terminology

Term	Meaning
Model	The engine-independent, serialisable description of a schema: tables, columns, constraints, plus layout
Dialect	A plug-in describing one engine/version: types, naming rules, capabilities, DDL syntax
Logical type	An engine-neutral type (e.g. `integer`, `string(n)`, `timestamp_tz`) that maps to a physical type per dialect
Issue	A validation result (error, warning or info) attached to a model element
Quick fix	A model command attached to an issue that resolves it when applied
Command	An undoable, serialisable change to the model (e.g. `AddColumn`, `MoveTable`)

2. Requirements

2.1 Functional requirements

ID	Requirement	Phase
FR-01	Create, rename, duplicate and delete tables; set schema and comment	1
FR-02	Add, edit, reorder and delete columns: name, data type and parameters, nullable, default, auto-increment/identity, comment	1
FR-03	Define primary keys, including composite keys	1
FR-04	Define foreign keys by dragging from a source column to a target column, or through a form; support composite FKs and `ON DELETE` / `ON UPDATE` actions	1
FR-05	Drag tables on the canvas; pan, zoom, fit-to-screen, minimap, auto-layout	1
FR-06	Undo/redo for every model change; autosave to local storage; open/save project files	1
FR-07	Select a target engine and version; type pickers show only that engine's types	2
FR-08	Convert a model between engines with type mapping and a conversion report	2
FR-09	Validate data types, naming rules, reserved words, identifier lengths and constraint requirements per engine	2
FR-10	Show issues on the canvas (table/column badges) and in a Problems panel, with explanation and quick fixes	3
FR-11	Display generated DDL live, syntax-highlighted, synchronised with the selection	4
FR-12	Export DDL to `.sql` (single file, per-table archive, or migration diff)	5
FR-13	Connect to a live database; list schemas/tables; reverse-engineer selected tables onto the canvas	6
FR-14	Browse and edit table data; import rows from Excel with column mapping; apply as DML in a transaction	7
FR-15	Show performance metrics of the connected database (connections, throughput, cache, locks, top queries)	8

2.2 Non-functional requirements

Category	Requirement
Responsiveness	Canvas interactions at 60 fps for 200 tables; usable up to 1,000 tables with viewport culling
Latency	Validation results under 150 ms after an edit for a 200-table model (incremental, off the UI thread)
Layout	Responsive from 360 px phones to 4K screens; panels collapse into drawers/bottom sheets below 900 px
Accessibility	WCAG 2.2 AA: keyboard operation of canvas and panels, screen-reader labels, non-colour severity cues
Reliability	No lost work: autosave every change (debounced) and crash-safe recovery from IndexedDB
Security	DB credentials never stored in or sent to the browser after entry; server-side vault; least-privilege, read-only by default
Portability	Model file format is versioned JSON with forward migrations
Extensibility	Engines, validation rules and exporters are registered plug-ins

3. Architecture overview

The system is a **TypeScript monorepo**. Phases 1–5 run entirely in the browser with no backend: modelling, validation and DDL generation are pure functions over the model. Phases 6–8 add a backend service, because browsers cannot open database connections and credentials must stay server-side.

The core design principle is a **single source of truth**: one immutable model, changed only through commands. Canvas, inspector, problems panel and DDL view are all projections of that model.

3.1 Component diagram

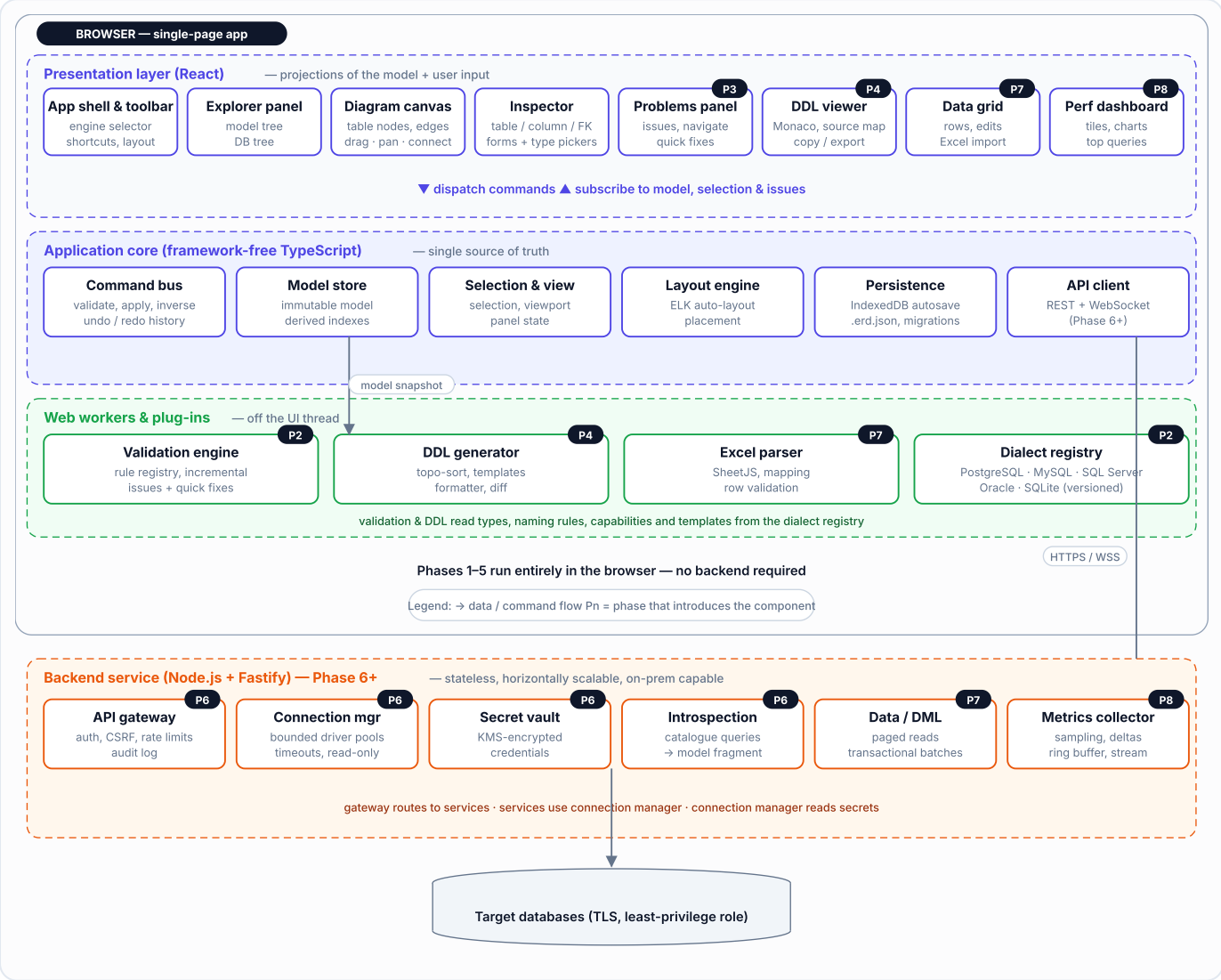


Figure 1 — Component diagram. UI components never change state directly: they dispatch commands to the core and re-render from the model store. Validation and DDL generation run in workers and read all engine knowledge from the dialect registry. The backend appears only in Phase 6, behind the API client.

3.2 Component responsibilities

Component	Responsibility	Key interfaces
App shell & toolbar	Layout of panels, engine selector, global actions, keyboard shortcuts, responsive breakpoints	dispatches commands; reads view state

Component	Responsibility	Key interfaces
Explorer panel	Searchable list of model tables (Model tab) and live database tree (Database tab, Phase 6)	`selectElement()`, `importTables()`
Diagram canvas	Renders table nodes and relationship edges; handles drag, pan, zoom, connect; viewport culling	emits `MoveTables`, `CreateForeignKey`; reads model, issues, selection
Inspector	Forms for the selected table/column/FK, with per-dialect type pickers and inline field errors	dispatches `Update*` commands
Problems panel	Grouped, sortable, filterable issue list; navigate-to-element; apply quick fix	reads issue set; dispatches quick-fix commands
DDL viewer	Live, syntax-highlighted DDL for whole model or selection; copy/export	reads generator output
Command bus	Validates and applies commands, records inverse commands for undo/redo, coalesces drags, publishes change events	`dispatch(cmd)`, `undo()`, `redo()`
Model store	Holds immutable model plus derived indexes (by id, by name, FK graph)	`getState()`, `subscribe()`
Validation engine	Runs rules incrementally on changed elements in a worker; returns issues with fixes	`validate(model, dialect, changedIds)`
DDL generator	Topologically orders tables, emits statements via dialect templates, formats output	`generate(model, dialect, options)`
Dialect registry	Loads dialect plug-ins; exposes types, naming rules, reserved words, capabilities, type mappings	`get(engine, version)`
Persistence adapter	Autosave to IndexedDB; open/save `.erd.json`; schema-version migrations	`save()`, `load()`, `migrate()`
Layout engine	Automatic layered layout (ELK.js) and collision-free placement of new tables	`autoLayout(model)`
Backend services	Connections, introspection, DML execution, Excel-to-DML, metrics (Phases 6–8)	REST + WebSocket API (§10)

3.3 Technology choices

Concern	Choice	Rationale
Language	TypeScript (strict) everywhere	One type system shared by UI, core, dialects and server
UI framework	React 19 + Vite	Mature ecosystem, fast dev server, good worker support
Canvas	React Flow (`@xyflow/react`) with custom table nodes	Built-in pan/zoom/minimap, per-column connection handles, viewport-only rendering
State	Zustand + Immer (structural sharing) behind a command bus	Small, predictable, easy undo via inverse commands
Components & styling	Radix UI primitives + Tailwind CSS, themed with the **Sunrise Studio** tokens (§9.4)	Accessible primitives; one token set drives every component and a later dark theme
Code view	Monaco editor (read-only mode)	SQL highlighting, folding, search, diff view for migrations

Concern	Choice	Rationale
Auto-layout	ELK.js (layered algorithm) in a worker	Good results for ER graphs; off the UI thread
Storage	IndexedDB via Dexie	Large models, crash recovery, offline use
Excel	SheetJS (parsing in a worker)	Reads `.xlsx` / `.xls` / `.csv` in the browser without uploads
Charts	uPlot / ECharts	Fast time series for the metrics dashboard
Backend	Node.js + Fastify, `pg`, `mysql2`, `tedious` (mssql), `oracledb`, `better-sqlite3`	Same language and dialect package on both sides
Testing	Vitest, Playwright (E2E + visual), Testcontainers (real engines)	DDL is verified by running it on real databases

3.4 Package structure

```
er-visual-designer/
├── packages/
│   ├── model/          # Domain types, JSON schema, commands, migrations
│   ├── dialects/       # Dialect SPI + postgres, mysql, mssql, oracle, sqlite
│   ├── validation/     # Rule engine + rule catalogue
│   ├── ddl/            # DDL generator, formatter, diff (migrations)
│   ├── web/            # React SPA (canvas, panels, workers)
│   ├── server/         # Fastify API (Phase 6+)
│   └── introspect/     # Catalogue queries per engine (Phase 6+)
└── e2e/                # Playwright tests
```

`model`, `dialects`, `validation` and `ddl` are pure, framework-free libraries with no DOM access. They run unchanged in the browser, in workers, on the server and in a future CLI.

4. Domain model

4.1 Class diagram

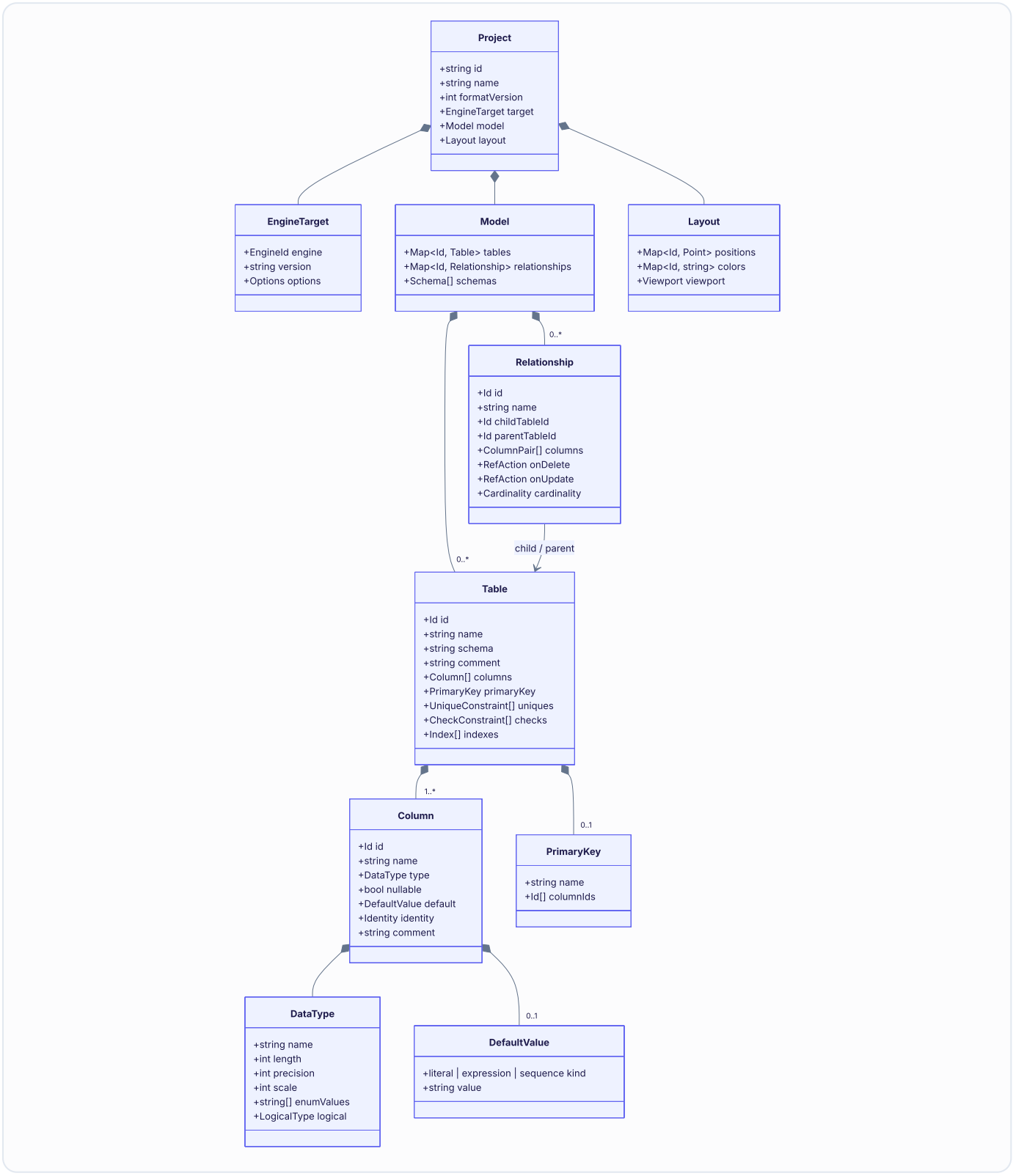


Figure 2 — Domain model. Elements reference each other by stable ID, never by name, so renames never break references.

4.2 Key modelling decisions

1. **Stable IDs, not names.** Tables, columns and relationships have UUIDs (ULIDs). FKs reference column IDs, so renaming a column updates every relationship and the DDL automatically.
2. **Layout is separate from the schema.** Positions, colours and viewport are stored in `Layout`, so moving a table is not a schema change. It does not trigger validation or DDL regeneration.
3. **Physical type plus logical type.** Every column stores the engine's physical type (e.g. `numeric(12,2)`) and its logical type (`decimal(12,2)`). The logical type makes engine conversion deterministic (see §5.3).
4. **Relationships are first-class.** An FK is a `Relationship` with ordered column pairs, so composite FKs and cardinality (1:1, 1:N, optional/mandatory) come from one structure.
5. **Constraint names are optional.** If none is given, the generator derives a deterministic name (`pk_<table>`, `fk_<child>_<parent>`, `uq_<table>_<cols>`) that respects the dialect's length limit, using a hash suffix when truncation is needed.

4.3 Serialised project format (`.erd.json`)

```
{
  "formatVersion": 1,
  "name": "shop_schema",
  "target": { "engine": "postgresql", "version": "16" },
  "model": {
    "tables": [{
      "id": "t_01HF...", "name": "orders", "schema": "public",
      "columns": [
        { "id": "c_01", "name": "id", "type": { "name": "bigint", "logical": "int64" },
          "nullable": false, "identity": { "kind": "always" } },
        { "id": "c_02", "name": "customer_id", "type": { "name": "bigint", "logical": "int64" }, "nullable": false
        { "id": "c_03", "name": "status", "type": { "name": "varchar", "length": 20, "logical": "string" },
          "nullable": false, "default": { "kind": "literal", "value": "pending" } }
      ],
      "primaryKey": { "name": "pk_orders", "columnIds": ["c_01"] }
    }],
    "relationships": [{
      "id": "r_01", "name": "fk_orders_customer",
      "childTableId": "t_01HF...", "parentTableId": "t_01HE...",
      "columns": [{ "child": "c_02", "parent": "c_10" }],
      "onDelete": "restrict", "onUpdate": "no_action"
    }]
  },
  "layout": { "positions": { "t_01HF...": { "x": 420, "y": 80 } }, "viewport": { "x": 0, "y": 0, "zoom": 1 } }
}
```

The file is validated against a JSON Schema on load. A `formatVersion` change comes with a migration function, so old files always open.

5. Database engine dialects

5.1 Dialect plug-in interface

Each engine is a plug-in that implements a single interface. Validation, the inspector's type pickers and DDL generation get all engine-specific knowledge from this interface, so the core never contains `if (engine === 'mysql')` branches.

```
interface Dialect {
  id: EngineId;                       // 'postgresql' | 'mysql' | 'mssql' | 'oracle' | 'sqlite'
  displayName: string;
  versions: string[];                 // e.g. ['13','14','15','16','17']
  identifiers: {
    maxLength: number;                // 63 PG, 64 MySQL, 128 SQL Server / Oracle 12.2+
    pattern: RegExp;                  // legal unquoted identifier
    caseFolding: 'lower' | 'upper' | 'preserve';
    quote(name: string): string;      // "x", `x`, [x]
    reservedWords: ReadonlySet<string>;
  };
  types: TypeDescriptor[];            // name, params (length/precision/scale ranges), category, aliases
  capabilities: {
    identity: 'generated' | 'auto_increment' | 'identity' | 'autoincrement';
    checkConstraints: boolean;
    deferrableConstraints: boolean;
    alterAddForeignKey: boolean;      // false for SQLite
    schemas: boolean;
    booleanType: boolean;
    onUpdateActions: RefAction[];
  };
  mapLogicalType(t: LogicalType): DataType | MappingWarning;
  toLogicalType(t: DataType): LogicalType;
  ddl: DdlTemplates;                  // createTable, column, pk, fk, unique, index, comment, drop
  rules?: ValidationRule[];           // engine-specific extra rules
}
```

5.2 Initial engine matrix

Capability	PostgreSQL 13–17	MySQL 8.x	SQL Server 2019+	Oracle 19c / 23ai	SQLite 3.x
Max identifier length	63 bytes	64 chars	128 chars	128 bytes	no limit
Unquoted case folding	lower	preserved (table names: OS-dependent)	preserved (collation)	upper	preserved
Identifier quoting	`"name"`	`` `name` ``	`[name]`	`"NAME"`	`"name"`
Auto-increment	`GENERATED … AS IDENTITY`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`GENERATED … AS IDENTITY`	`INTEGER PRIMARY KEY [AUTOINCREMENT]`
Native boolean	yes	`BOOLEAN` = `TINYINT(1)`	`BIT`	23ai only	stored as integer
Add FK after create	yes	yes	yes	yes	**no** — FKs must be inline

Capability	PostgreSQL 13–17	MySQL 8.x	SQL Server 2019+	Oracle 19c / 23ai	SQLite 3.x
Schemas	yes	database = schema	yes	user = schema	attached DBs only

The matrix is the starting point for the dialect plug-ins. Each value is backed by an automated test against the real engine (§12).

5.3 Logical type mapping

Converting a model between engines maps each physical type to its logical type, then to the target engine's physical type:

Logical	PostgreSQL	MySQL	SQL Server	Oracle	SQLite
`int32`	`integer`	`INT`	`INT`	`NUMBER(10)`	`INTEGER`
`int64`	`bigint`	`BIGINT`	`BIGINT`	`NUMBER(19)`	`INTEGER`
`decimal(p,s)`	`numeric(p,s)`	`DECIMAL(p,s)`	`DECIMAL(p,s)`	`NUMBER(p,s)`	`NUMERIC`
`string(n)`	`varchar(n)`	`VARCHAR(n)`	`NVARCHAR(n)`	`VARCHAR2(n CHAR)`	`TEXT`
`text`	`text`	`LONGTEXT`	`NVARCHAR(MAX)`	`CLOB`	`TEXT`
`boolean`	`boolean`	`BOOLEAN`	`BIT`	`BOOLEAN` (23ai) / `NUMBER(1)`	`INTEGER`
`timestamp_tz`	`timestamptz`	`TIMESTAMP` ⚠	`DATETIMEOFFSET`	`TIMESTAMP WITH TIME ZONE`	`TEXT` ⚠
`uuid`	`uuid`	`BINARY(16)` / `CHAR(36)`	`UNIQUEIDENTIFIER`	`RAW(16)`	`TEXT`
`json`	`jsonb`	`JSON`	`NVARCHAR(MAX)` ⚠	`JSON` (21c+)	`TEXT`

⚠ marks lossy mappings. Each one produces a conversion warning that explains what is lost (for example, MySQL `TIMESTAMP` stores UTC and has a 2038 range limit).

6. Validation engine

6.1 Design

Validation is a set of small, independent **rules**. Each rule declares its scope (model, table, column or relationship) and the dialects it applies to. It is a pure function that returns zero or more issues.

```
interface ValidationRule {
  id: string;                        // e.g. 'E-TYP-001'
  scope: 'model' | 'table' | 'column' | 'relationship';
  appliesTo?: EngineId[];            // undefined = all engines
  defaultSeverity: 'error' | 'warning' | 'info';
  check(ctx: RuleContext): Issue[];
}

interface Issue {
  ruleId: string;
  severity: 'error' | 'warning' | 'info';
  target: { kind: 'table' | 'column' | 'relationship' | 'model'; id: Id; field?: string };
  message: string;                   // one line: what is wrong
  explanation: string;               // why it matters for this engine, with doc link
  fixes: QuickFix[];                 // each fix = label + Command[] (undoable)
}
```

- **Incremental.** The command bus reports which element IDs changed. The engine re-runs only rules whose scope touches those elements, plus their FK neighbours. A full run happens on engine change or file load.
- **Off the main thread.** Rules run in a dedicated Web Worker, so typing in the inspector never stutters.
- **Configurable.** Users can change warning/info severities or suppress them per project (`.erd.json` → `validation.overrides`). Errors that would make the DDL fail cannot be suppressed.
- **Gating.** Errors block DDL export (Phase 5) and apply-to-database (Phase 7). Warnings don't.

6.2 Rule catalogue (initial)

Rule ID	Severity	Checks	Suggested fix
`E-NAM-001`	error	Name is empty	Generate name (`table_1` , `column_1`)
`E-NAM-002`	error	Duplicate table name in schema / duplicate column name in table (respecting case-folding)	Append suffix `_2`
`E-NAM-003`	error	Identifier longer than the engine maximum	Truncate with stable hash suffix
`E-NAM-004`	error / warn	Reserved word used as identifier (error if the dialect forbids it even quoted, otherwise warning)	Rename (e.g. `order` → `orders`) or quote always
`W-NAM-005`	warning	Characters require quoting (spaces, mixed case on PG/Oracle)	Convert to `snake_case`
`E-TYP-001`	error	Data type not supported by engine/version	Map via logical type
`E-TYP-002`	error	Type parameter missing or out of range (e.g. `VARCHAR` without length on MySQL, `DECIMAL(40,2)` on SQL Server > 38)	Clamp to the allowed range
`E-DEF-001`	error	Default value not valid for the type (e.g. `'abc'` on `integer`)	Remove default / convert literal

Rule ID	Severity	Checks	Suggested fix
`E-DEF-002`	error	Default form not allowed for type on engine (e.g. literal default on MySQL `TEXT` / `BLOB` / `JSON`)	Use expression default `('…')` (MySQL 8.0.13+) or remove
`E-PK-001`	error	Primary-key column is nullable	Set NOT NULL
`W-PK-002`	warning	Table has no primary key	Add `id` identity column as PK
`E-IDN-001`	error	Identity/auto-increment on non-integer type, or more than one per table (MySQL, SQL Server)	Change type to `bigint`
`E-FK-001`	error	FK column count differs from referenced key	Open FK editor
`E-FK-002`	error	FK column type is incompatible with the referenced column	Change child type to match parent
`E-FK-003`	error	Referenced columns are not a PK or unique constraint	Add unique constraint on parent columns
`W-FK-004`	warning	`ON DELETE SET NULL` on a NOT NULL column	Make column nullable / change action
`E-FK-005`	error	Referential action not supported (e.g. `ON UPDATE` on Oracle)	Remove action
`I-IDX-001`	info	FK columns have no supporting index (needed for join and delete performance)	Add index
`I-REL-001`	info	Circular FK dependency (handled by generator, but worth knowing)	—
`W-TBL-001`	warning	Table has no columns	Add `id` column

6.3 Issue presentation

- **Canvas:** a severity badge on the table header (highest severity wins) and a coloured underline on the offending column. The edge is dashed red for a broken relationship. Hovering shows the message.
- **Inspector:** the offending field gets an inline error with the explanation and fix buttons.
- **Problems panel:** grouped by table or by rule. It can be sorted by severity and filtered by text. Clicking an issue selects and centres the element. "Fix" applies the quick fix as one undoable command, and "Fix all of this kind" applies it as one batch.
- **Accessibility:** severities use icon shape and text as well as colour (✖ error, ▲ warning, ● info).

7. State management, commands and undo/redo

All changes go through the **command bus**:

1. A UI component dispatches a command, e.g. `UpdateColumn { tableId, columnId, patch: { nullable: false } }`.
2. The bus checks structural invariants (target exists, IDs unique). It does not run engine rules; those produce issues, not rejections. This lets users work through intermediate invalid states.
3. The bus applies the command with Immer, producing the next immutable model and an inverse command.
4. The bus pushes `{ command, inverse }` onto the history stack and emits a `ModelChanged { changedIds }` event.
5. Subscribers react: the canvas re-renders affected nodes, the validator and DDL generator recompute (debounced 100 ms), and persistence autosaves (debounced 500 ms).

Coalescing. A drag emits many position updates. The canvas holds transient positions locally and dispatches one `MoveTables` command on drop, so undo returns the table to where it was before the drag. Continuous typing in one field is merged into a single history entry.

Transactions. Compound actions such as "create FK and create missing child column", or "apply all quick fixes", are wrapped in a `Batch` command, so they undo in one step.

Future collaboration. Commands are serialisable, so the same log can later be synchronised between users (e.g. through a CRDT/Yjs layer) without changing UI code.

8. DDL generation

8.1 Pipeline

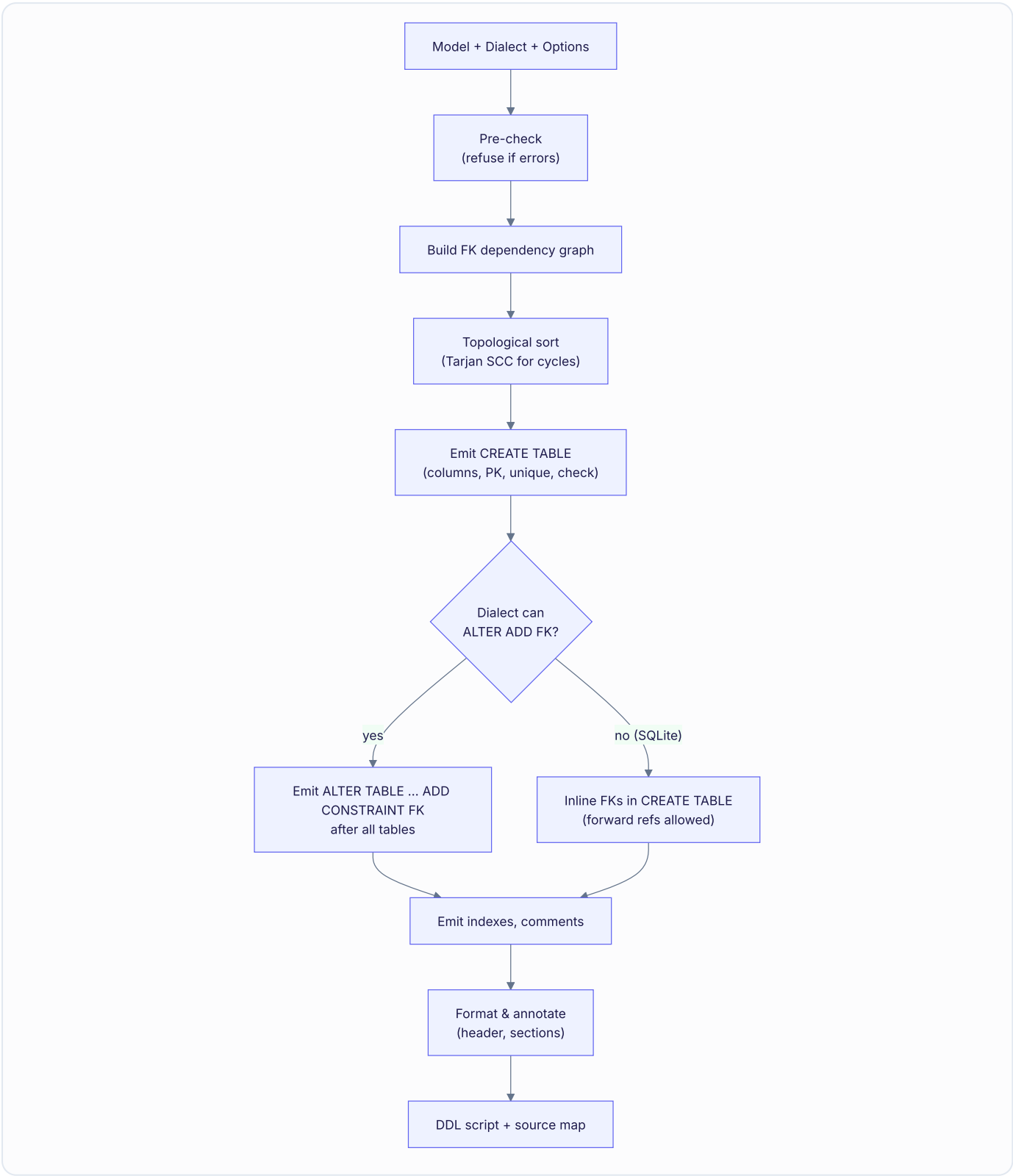


Figure 3 — DDL generation pipeline.

- **Ordering.** Tables are emitted in dependency order. FKs are emitted as separate `ALTER TABLE` statements after all tables by default, so cyclic references (e.g. `employees.manager_id → employees.id`, or A↔B) always work. An option inlines FKs when the graph has no cycles.
- **Quoting.** Identifiers are quoted only when needed (reserved word, special characters, case-folding conflict), or always if the user chooses.
- **Source map.** Each statement records the model element IDs it came from. Selecting a table highlights its DDL, and a DDL line can jump back to the canvas.
- **Options.** Include `DROP … IF EXISTS`, wrap in a transaction (where DDL is transactional), include comments, target schema, statement terminator (`;` or `GO`).
- **Migrations (Phase 5).** A diff of two model snapshots produces `ALTER` statements instead of `CREATE` statements.

8.2 Example output (same model, two engines)

```
-- PostgreSQL 16
CREATE TABLE "orders" (
  "id"          bigint GENERATED ALWAYS AS IDENTITY,
  "customer_id" bigint NOT NULL,
  "status"      varchar(20) NOT NULL DEFAULT 'pending',
  "total"       numeric(12,2),
  CONSTRAINT "pk_orders" PRIMARY KEY ("id")
);
ALTER TABLE "orders" ADD CONSTRAINT "fk_orders_customer"
  FOREIGN KEY ("customer_id") REFERENCES "customers" ("id") ON DELETE RESTRICT;
CREATE INDEX "ix_orders_customer_id" ON "orders" ("customer_id");

-- SQL Server 2022
CREATE TABLE [orders] (
  [id]          BIGINT IDENTITY(1,1) NOT NULL,
  [customer_id] BIGINT NOT NULL,
  [status]      NVARCHAR(20) NOT NULL CONSTRAINT [df_orders_status] DEFAULT N'pending',
  [total]       DECIMAL(12,2) NULL,
  CONSTRAINT [pk_orders] PRIMARY KEY ([id])
);
GO
ALTER TABLE [orders] ADD CONSTRAINT [fk_orders_customer]
  FOREIGN KEY ([customer_id]) REFERENCES [customers] ([id]);
GO
```

9. User interface design

9.1 Chosen visual direction — Sunrise Studio

> **Decision (8 November 2026):** of the three interactive mockups in `mockups/` (A Aurora Glass, B Sunrise Studio, C Neon Flow), **Option B — Sunrise Studio** is the visual direction for the product. Options A and C are kept for reference only.

Sunrise Studio is a **light, docked, IDE-style workspace**. A warm coral → magenta → violet gradient marks the brand and the primary actions; everything else stays neutral, so the schema itself is the focus.

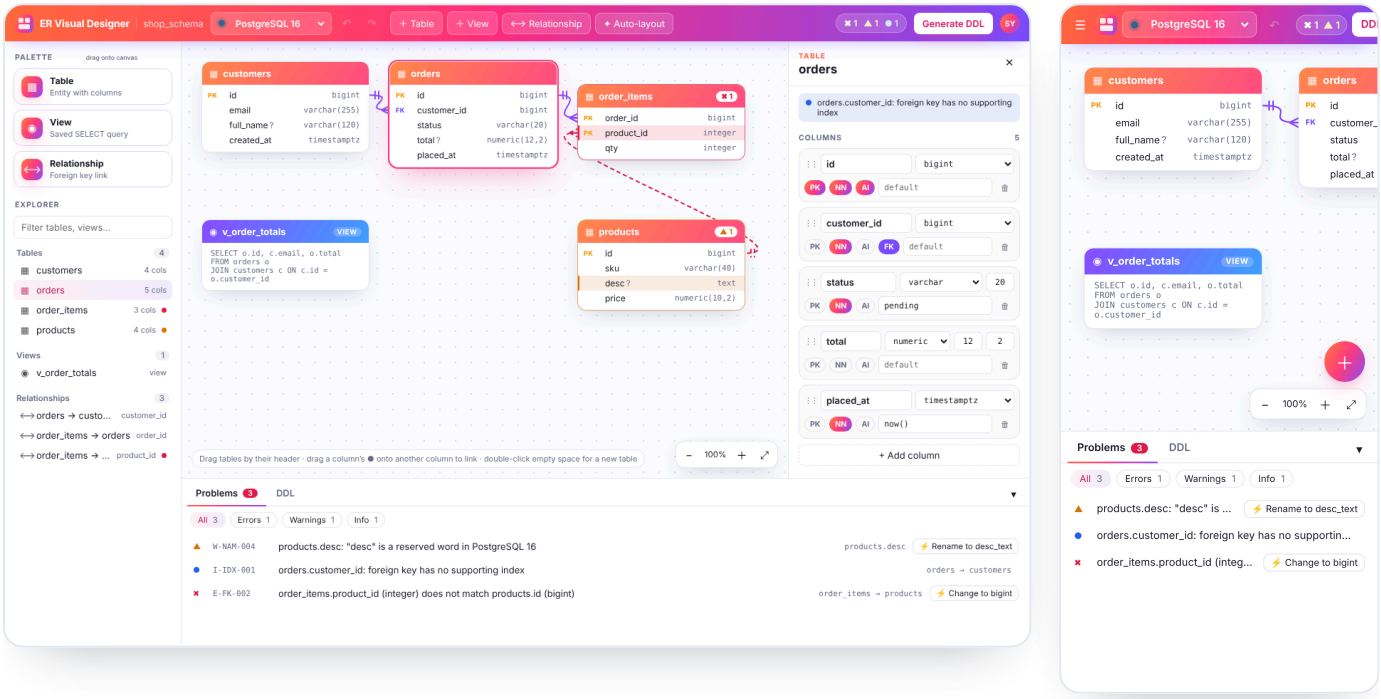


Figure 4 — Sunrise Studio (interactive mockup `mockups/option-b-sunrise-studio.html`). Desktop: gradient top bar with the database selector, palette and explorer (left), canvas, docked table editor (right), and the Problems/DDL panel (bottom). Phone: the explorer becomes a drawer, the editor a bottom sheet, and a floating "+" adds a table.

Why this direction

Criterion	Sunrise Studio
Readability	Highest contrast of the three options: dark text on white surfaces, and type names in monospace
Familiarity	Docked layout like VS Code, DBeaver or DataGrip, so there is little to learn
Documentation	Diagrams and screenshots print and paste into documents cleanly (light background)
Brand	The gradient gives a distinctive identity without tinting the whole UI
Trade-off accepted	A dark theme is a separate deliverable (derived from the same tokens, §9.4); it is not the default

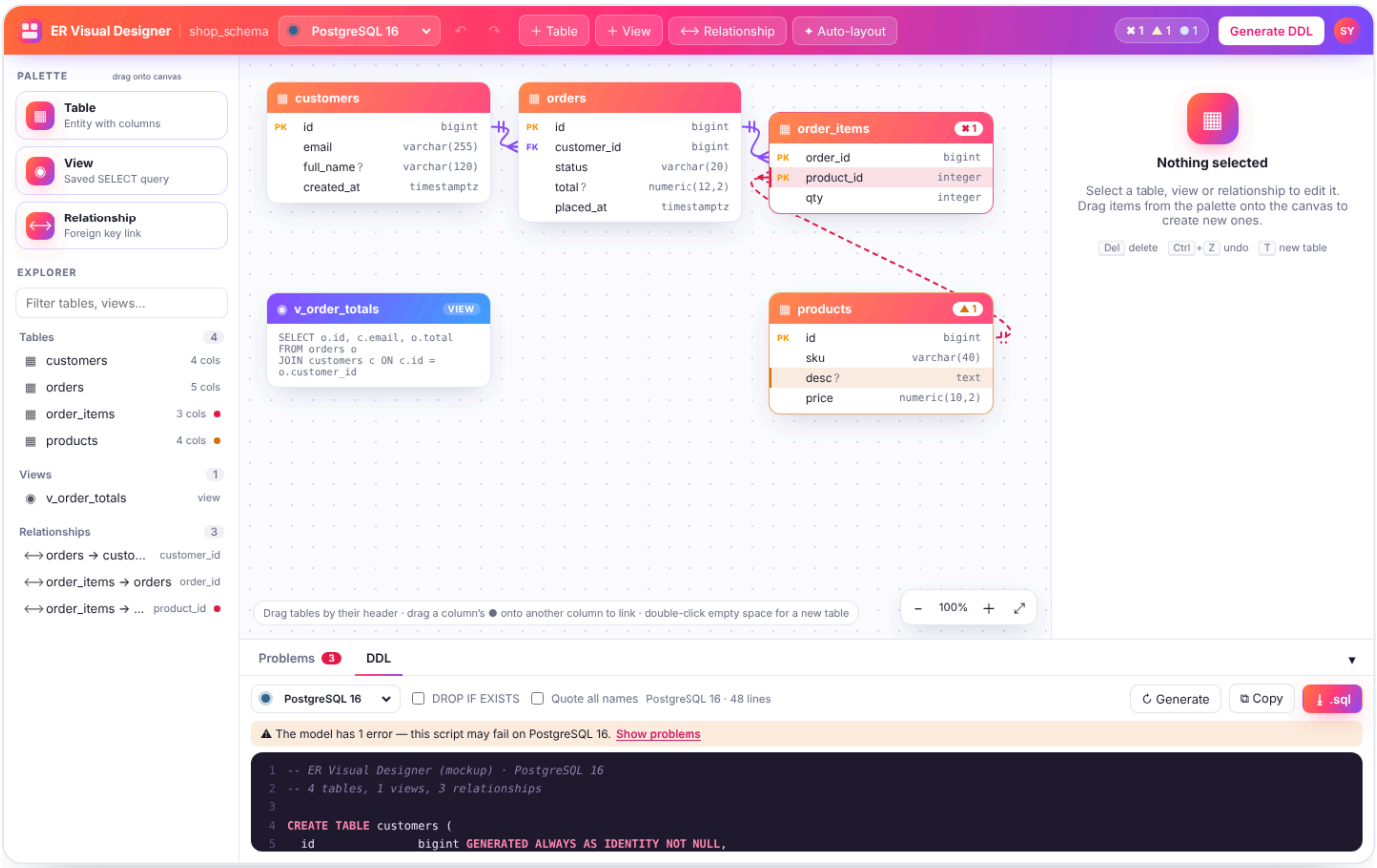


Figure 5 — DDL tab: the database selector is repeated in the tab toolbar, with options (DROP IF EXISTS, quote all names), Generate / Copy / Download .sql, a warning banner while errors exist, and a dark syntax-highlighted code panel.

9.2 Workspace layout

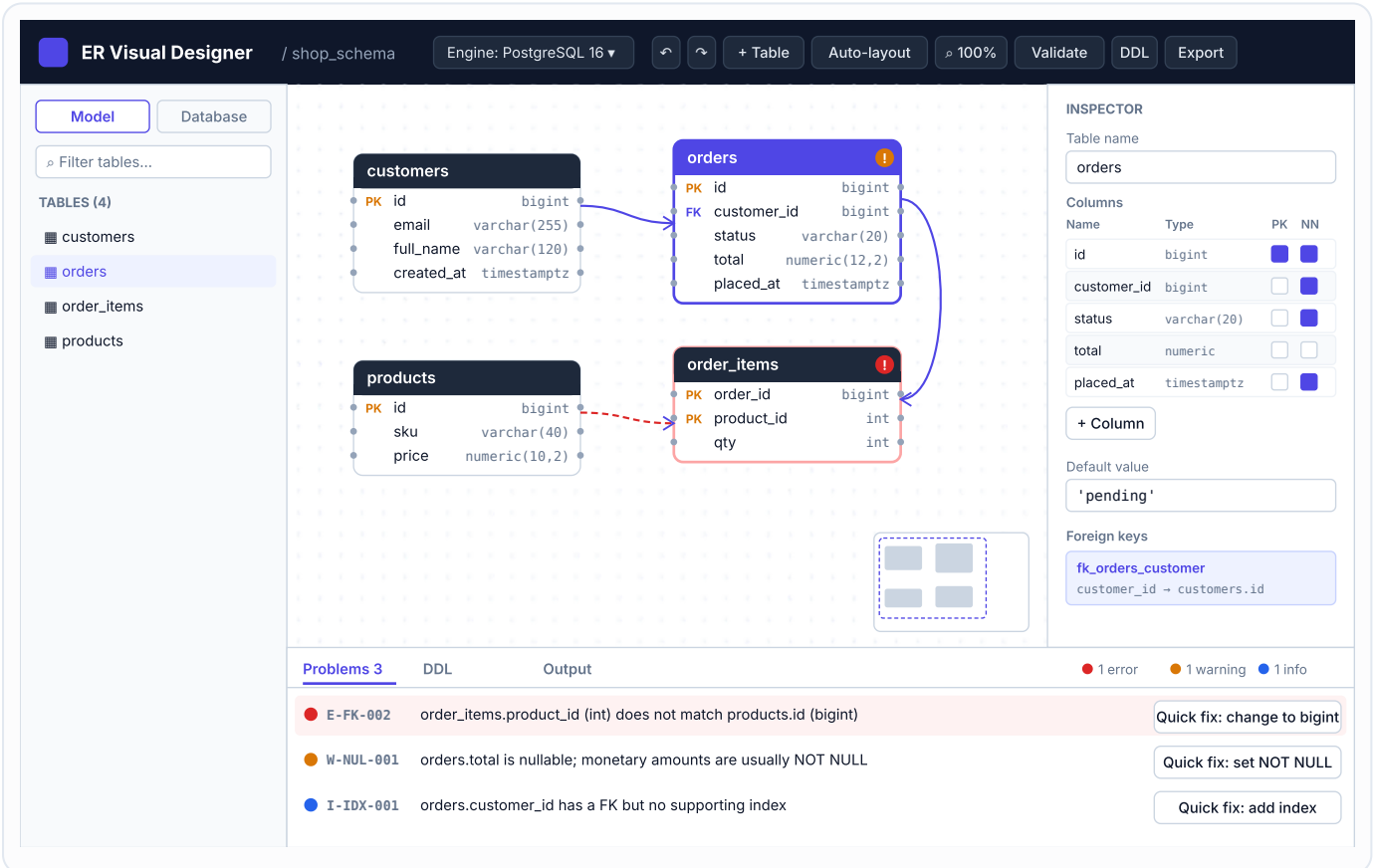


Figure 6 — Structural wireframe of the workspace (regions and behaviour); the visual styling comes from §9.4–9.5.

Region	Contents	Behaviour
Toolbar	Project name, engine/version selector, undo/redo, add table, auto-layout, zoom, validate, DDL, export	Collapses to an overflow menu below 1,200 px
Explorer (left)	Model tab: filterable table list. Database tab (Phase 6): connection tree	Resizable, collapsible; becomes a drawer below 900 px
Canvas (centre)	Table cards with typed columns, PK/FK markers, connection handles on each column, relationship edges with cardinality markers	Pan (space-drag / two-finger), zoom (wheel / pinch), marquee select, snap-to-grid
Inspector (right)	Table properties, column grid (name, type, PK, NN, default), FK list and editor	Becomes a bottom sheet on tablets and phones
Bottom panel	Problems, DDL, Output (later: Data grid)	Resizable, collapsible; badge counts on tabs

Region	Default size	Notes
Top bar	48 px high	Full-width brand gradient. Holds the database selector, undo/redo, add Table/View/Relationship, auto-layout, the problems chip and **Generate DDL**
Left panel	248 px (220 px below 1,280 px)	Palette (drag sources) above the Explorer tree
Right panel (editor)	340 px (310 px below 1,280 px)	Docked, scrolls on its own; sticky header with the element name
Bottom panel	236 px, collapsible to 42 px	Tabs: **Problems** (count badge) and **DDL**. Later phases add Data and Output

9.3 Key interactions

Action	Mouse / touch	Keyboard
Add table	Toolbar "+ Table", or double-click empty canvas	`T`
Add column	"+ Column" in inspector or on hover of table card	`Enter` on last column row
Rename	Double-click table header / column name	`F2`
Toggle PK / NOT NULL	Checkbox in inspector	`P` / `N` on focused column
Create FK	Drag from a column handle to a column on another table	`Ctrl+K` opens FK dialog
Move tables	Drag header; multi-select with marquee or Shift-click	Arrow keys (Shift = 10× step)
Delete	Context menu → Delete	`Delete`
Undo / redo	Toolbar	`Ctrl+Z` / `Ctrl+Shift+Z`
Jump to problem	Click problem row	`F8` / `Shift+F8` next / previous

9.4 Design tokens

All styling comes from one token set (implemented as CSS custom properties and mapped into the Tailwind theme). The mockup values are listed with the **production values** where the mockup colour fails WCAG 2.2 AA contrast (see §9.6).

Brand gradient



Token	Mockup value	Production value	Used for
`--grad-brand`	`#ff6a3d → #ff2e7e → #7a4dff` (100°)	`#c2410c → #d6246e → #6d3ff0`	Top bar, logo, avatar
`--grad-accent`	`#ff7a3d → #ff3d7f → #8a4dff` (120°)	`#c2410c → #d6246e → #6d3ff0`	Primary buttons, toggles that are on, palette icons, active tab underline
`--grad-table`	`#ff8a4c → #ff4d82`	`#c2410c → #be123c`	Table card headers
`--grad-view`	`#8a4dff → #3fa2ff`	`#6d3ff0 → #1d4ed8`	View card headers

Neutrals and surfaces

Token	Value	Used for
`--text`	`#1c1d2b`	Body text (16.7:1 on white)
`--muted`	`#6b7088`	Secondary text, type names (4.9:1)
`--bg`	`#f4f5fa`	App background
`--panel`	`#ffffff`	Panels, table cards, inputs
`--canvas`	`#fafbff` + dot grid `#d9ddea` (22 px)	Diagram canvas
`--line`	`#e7e8f0`	Borders and dividers
`--hover` / `--active`	`#f5f3ff` / `#fff0f5 → #f3efff`	Hover rows; selected explorer item
`--code-bg`	`#1e1b2e`	DDL panel (text `#ece9ff`, keywords `#ff8ab3`, strings `#ffd479`, numbers `#9fd6ff`, comments `#8b86ad`)

Semantic colours

Token	Mockup	Production	Used for
`--error`	`#e11d48`	`#be123c`	Errors, broken relationship lines, error badges
`--warning`	`#d97706`	`#b45309`	Warnings, warning badges
`--info`	`#2563eb`	`#2563eb`	Info issues
`--success`	`#059669`	`#047857`	"No problems" state
`--pk`	`#f59e0b`	`#b45309`	PK marker on columns
`--fk` / `--relationship`	`#7a4dff`	`#6d3ff0`	FK marker, relationship lines and crow's-foot ends

Type, shape and depth

Token	Value
Font (UI)	Inter, falling back to Segoe UI / system-ui. 13 px base; 14 px for the brand; 17 px for the editor title
Font (code, types)	JetBrains Mono, falling back to Consolas / Menlo. 11–12 px
Section labels	10.5 px, uppercase, 0.09 em letter-spacing, muted
Radius	8 px for controls, 12 px for cards and table nodes, 999 px for pills and chips; panels are square (docked)
Spacing	4 px base scale (4 / 8 / 12 / 14 / 16 / 24)
Elevation	Table card `0 10px 28px -14px rgba(40,30,90,.28)`; selected card adds a 2 px accent ring plus a pink glow; floating controls `0 10px 28px -12px rgba(40,30,90,.3)`
Motion	120–250 ms ease for hover, drawer and sheet; a 1 s pulse when the user jumps to an element from Problems

9.5 Component styling

Component	Styling (Sunrise Studio)
Top bar	Brand gradient with white text. Controls are translucent white (`rgba(255,255,255,.16)` , 1 px `rgba(255,255,255,.3)` border). **Generate DDL** is the only solid white button, with pink text, so it stands out
Database selector	Pill with a coloured dot for the engine (PostgreSQL blue, MySQL teal, SQL Server red, Oracle red, SQLite blue), shown in the top bar and repeated in the DDL tab
Palette items	White cards with a gradient icon tile, title and hint. They lift 1 px with a pink glow on hover, and show a grab cursor. Clicking or tapping adds the item at the centre of the view (for touch)
Explorer	Sections for Tables, Views and Relationships with count pills. The selected item uses the active gradient wash and pink text. A red or amber dot marks elements with errors or warnings
Table card	236 px wide with a gradient header (name and severity badge) and a white body. Each row shows a PK/FK marker, the name (with `?` when nullable) and the monospace type. Connection dots appear on hover at both ends of every row. Rows with problems get a tinted background and a coloured left edge
View card	Same card shape with the violet→blue header and a `VIEW` tag; the body shows the first lines of the SELECT
Relationship line	2 px violet curve with crow's foot (many) at the child and a double bar (one) at the parent. Errors show as dashed red; the selected line is thicker with a glow
Table editor	Sticky header (gradient "TABLE" kicker and an editable title). Each column is a card: name, grouped type dropdown and size inputs on row 1; **PK / NN / AI** pill toggles (gradient when on), an FK pill, default value and delete on row 2. "+ Add column" is a dashed full-width button
Problems tab	Filter pills (All / Errors / Warnings / Info with counts). Each row shows the severity icon, rule code (monospace), message, location and a ⚡ **quick fix** button. Clicking a row selects the element and pulses it
DDL tab	Toolbar (database selector, options, Generate / Copy / **.sql** download) above the dark code panel with line numbers. An amber banner appears while the model has errors
Toast	Dark pill at the bottom centre that confirms actions ("Linked …", "Fixed …"), with an undo hint for deletes

9.6 Accessibility of the palette

Contrast was measured against WCAG 2.2 AA (4.5:1 for normal text, 3:1 for large/bold ≥ 14 pt and for UI graphics):

Pair	Mockup ratio	Result	Production fix
White text on gradient start `#ff6a3d`	2.85	✖ fails	Gradient start `#c2410c` (5.18)
White text on gradient middle `#ff2e7e`	3.53	✖ fails (normal text)	Middle `#d6246e` (4.84)
White text on gradient end `#7a4dff`	4.85	✓	End `#6d3ff0` (5.80)
White on table header `#ff8a4c`	2.34	✖ fails	Header gradient `#c2410c → #be123c` (5.18 / 6.29)
PK marker `#f59e0b` on white	2.15	✖ fails	`#b45309` (5.02)
Warning `#d97706` on white	3.19	✖ fails (text)	`#b45309` (5.02)
Success `#059669` on white	3.77	✖ fails (text)	`#047857` (5.48)
Body text, muted text, error, info, link, FK	4.7–16.7	✓	—
DDL code colours on `#1e1b2e`	7.6–14.1	✓	—

The production values keep the same hues and the same coral→magenta→violet direction, a little deeper. Severity is always shown with an icon and text (✖ ▲ ●) as well as colour.

9.7 Responsive behaviour

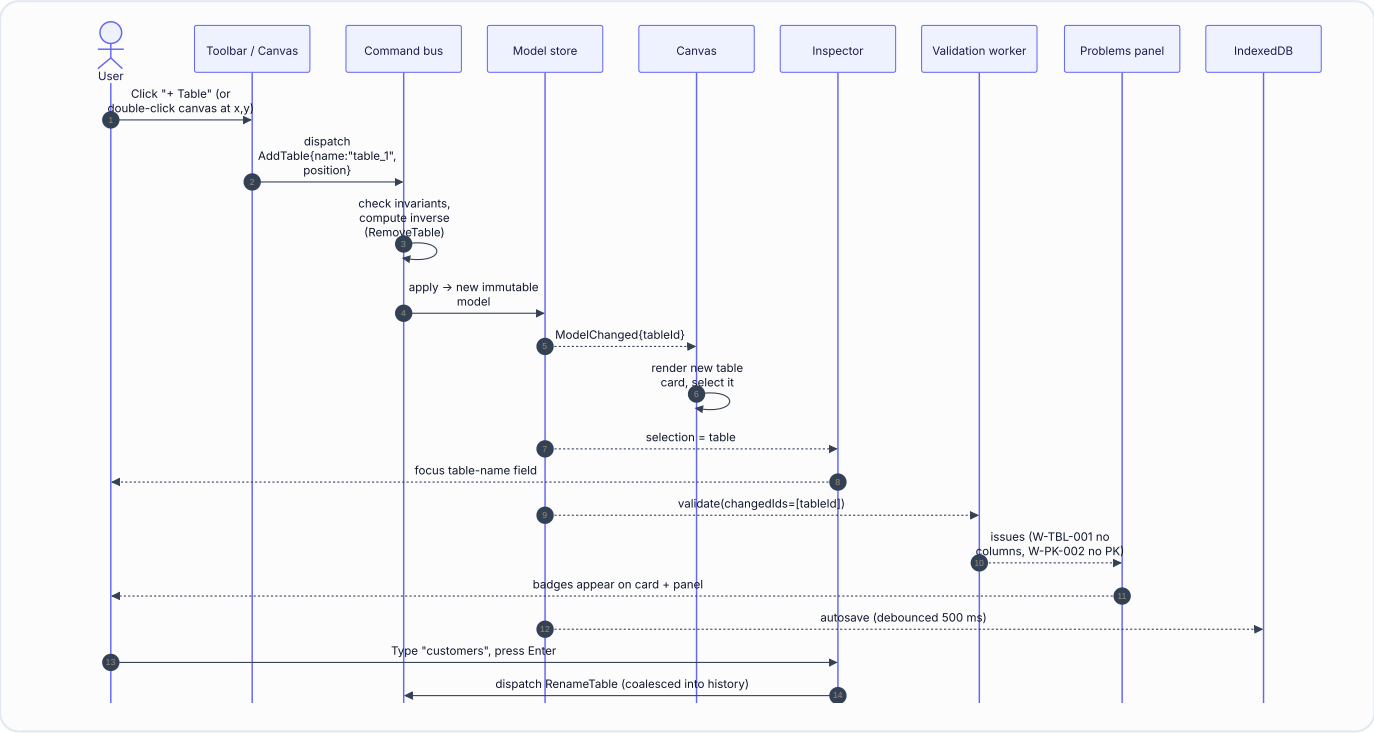
Width	Layout
≥ 1,280 px	Full layout: left 248 px, right 340 px, bottom 236 px, all toolbar actions visible
900–1,279 px	Side panels narrow to 220 / 310 px; auto-layout moves into the overflow menu
< 900 px (tablet, phone)	Single column (canvas above the bottom panel). ☰ opens the palette and explorer as a left drawer; selecting an element opens the editor as a bottom sheet (72 % height). Floating "+" adds a table. Toolbar shows the database selector, undo, the problems chip and **DDL**
< 480 px	Brand name and info count hidden; the problems list shows only icon, message and fix

Touch: tap a palette item to add it, drag table headers to move tables, drag a column dot to link, and use pinch or the zoom buttons to zoom.

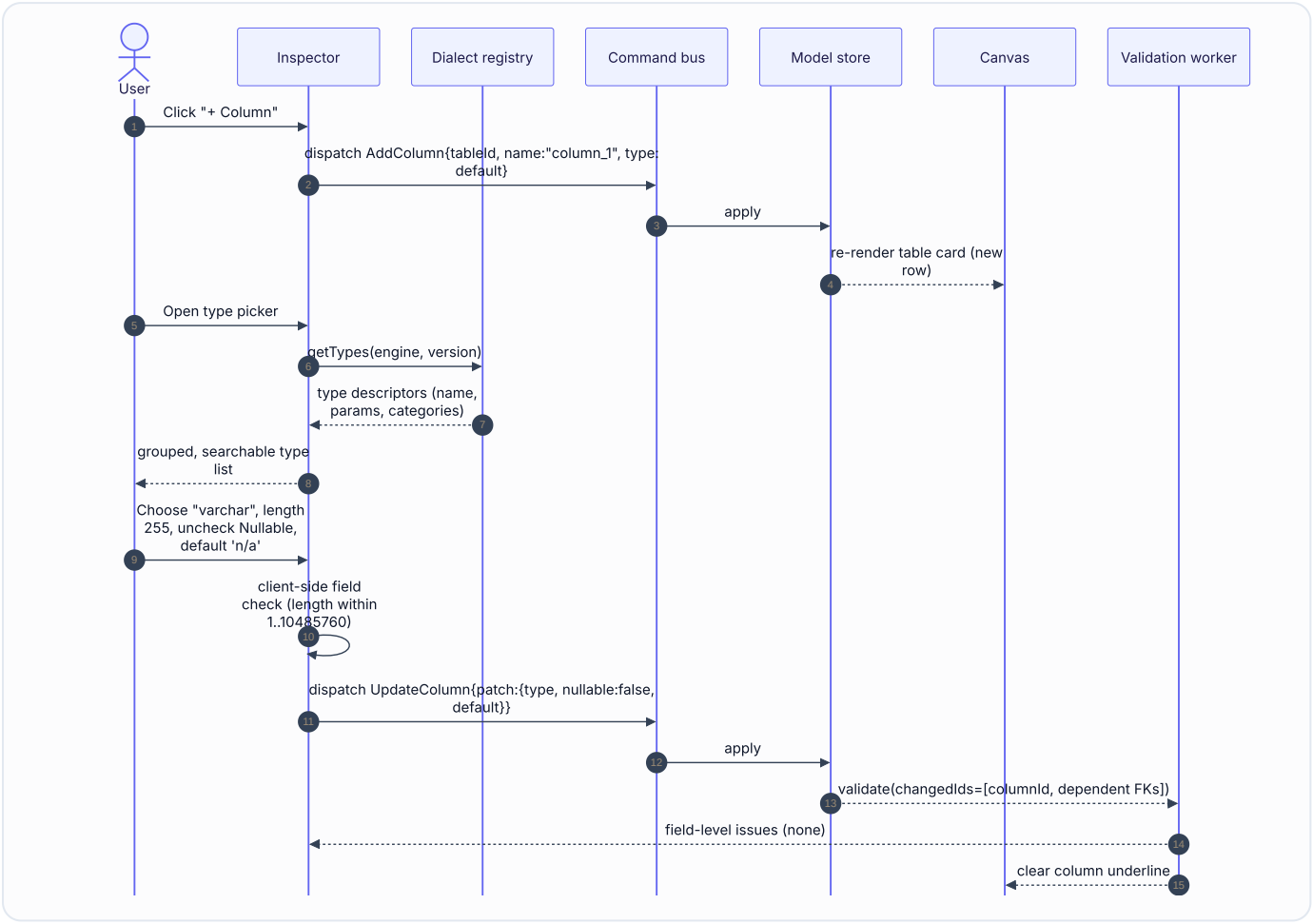
10. UI interaction sequences

The diagrams below show the main user interactions end to end. In all of them, *UI* means the React component the user is working with, and **Bus** is the command bus.

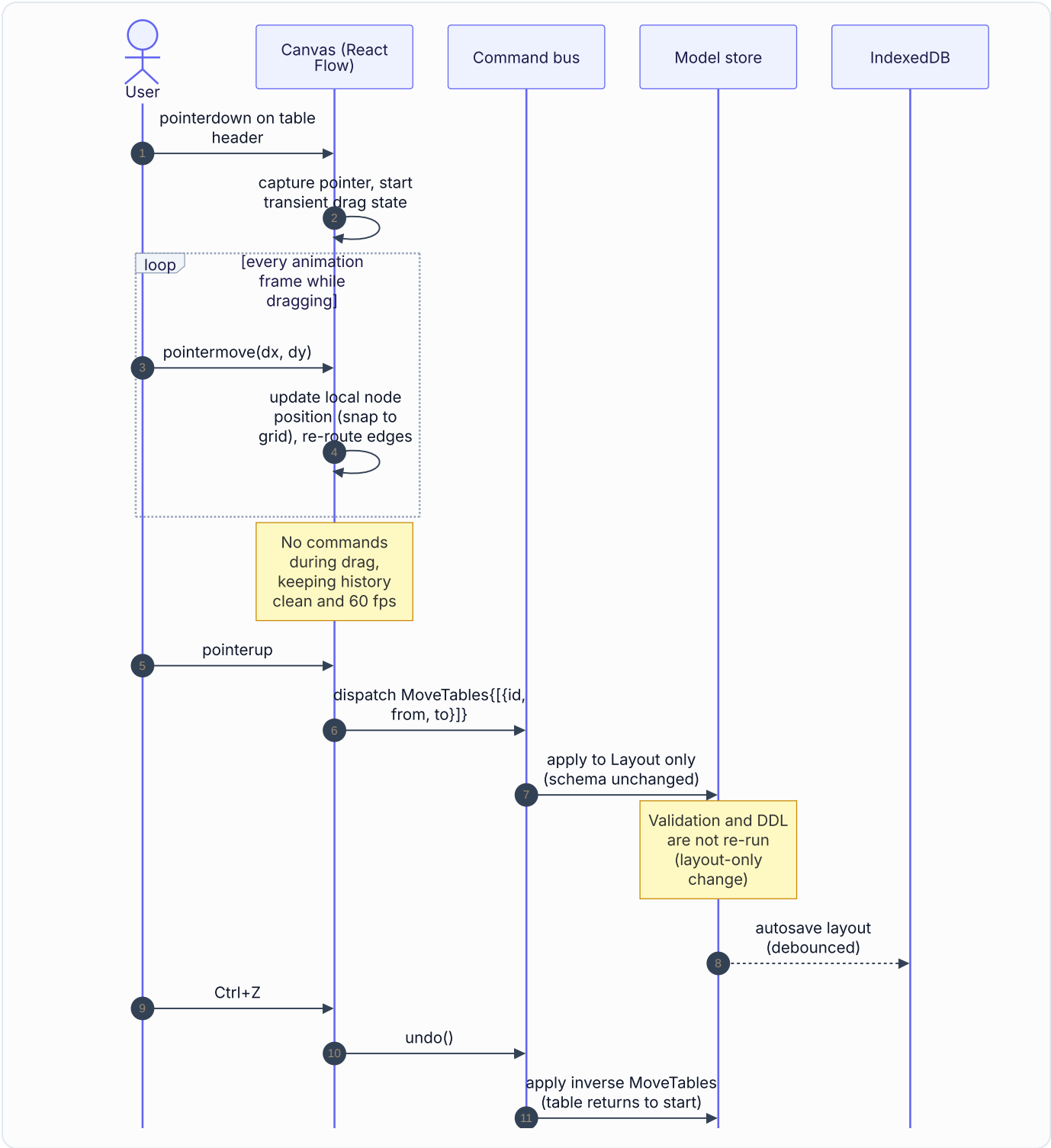
10.1 Create a table



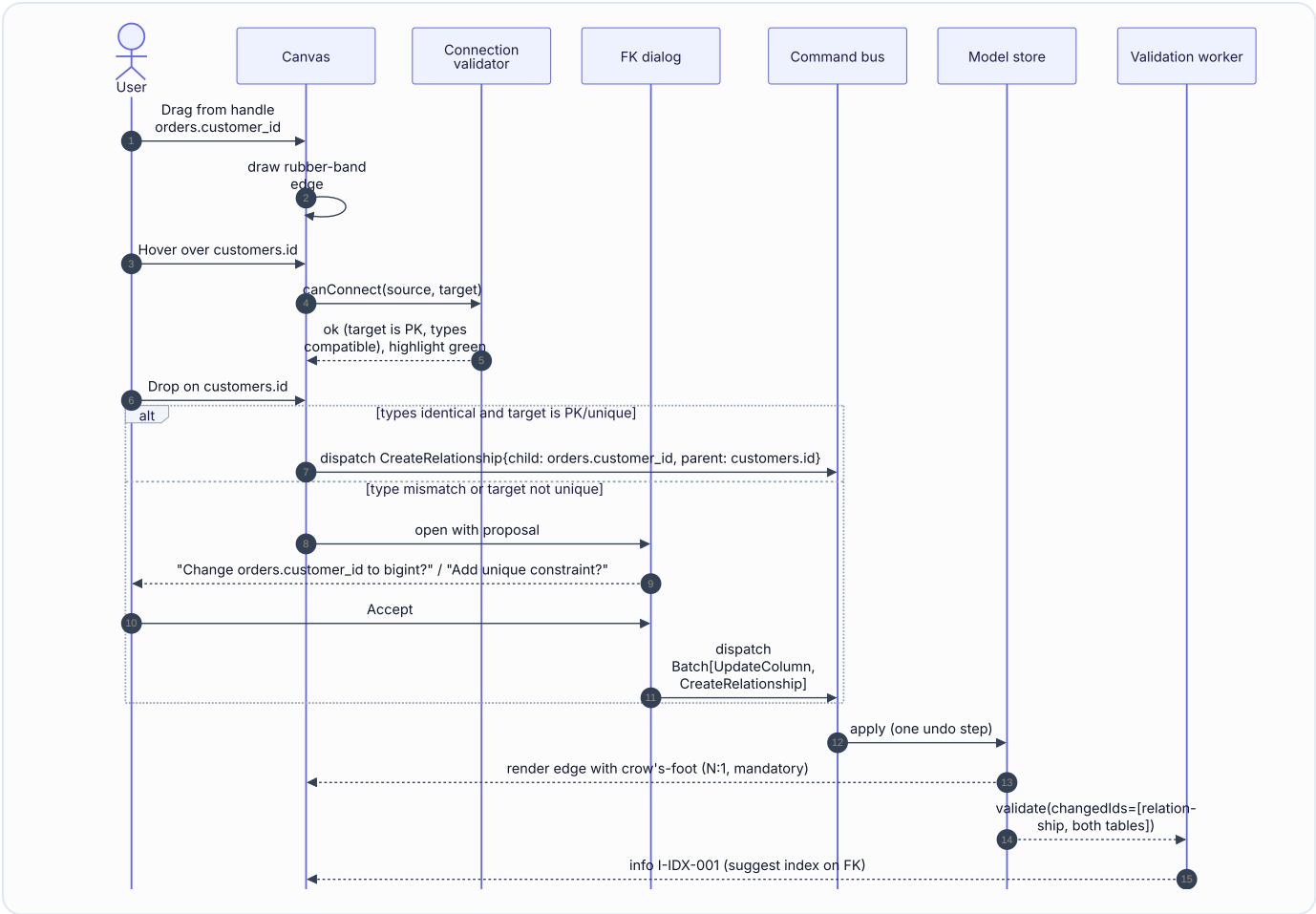
10.2 Add and edit a column



10.3 Drag a table on the canvas

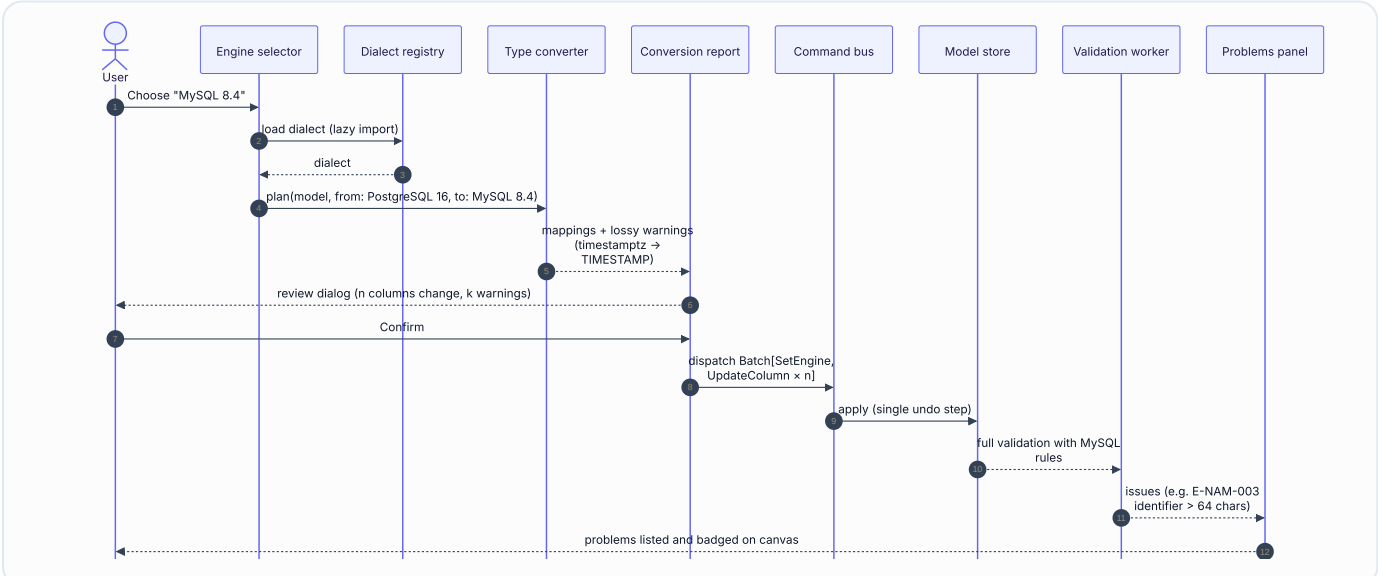


10.4 Draw a foreign-key relationship

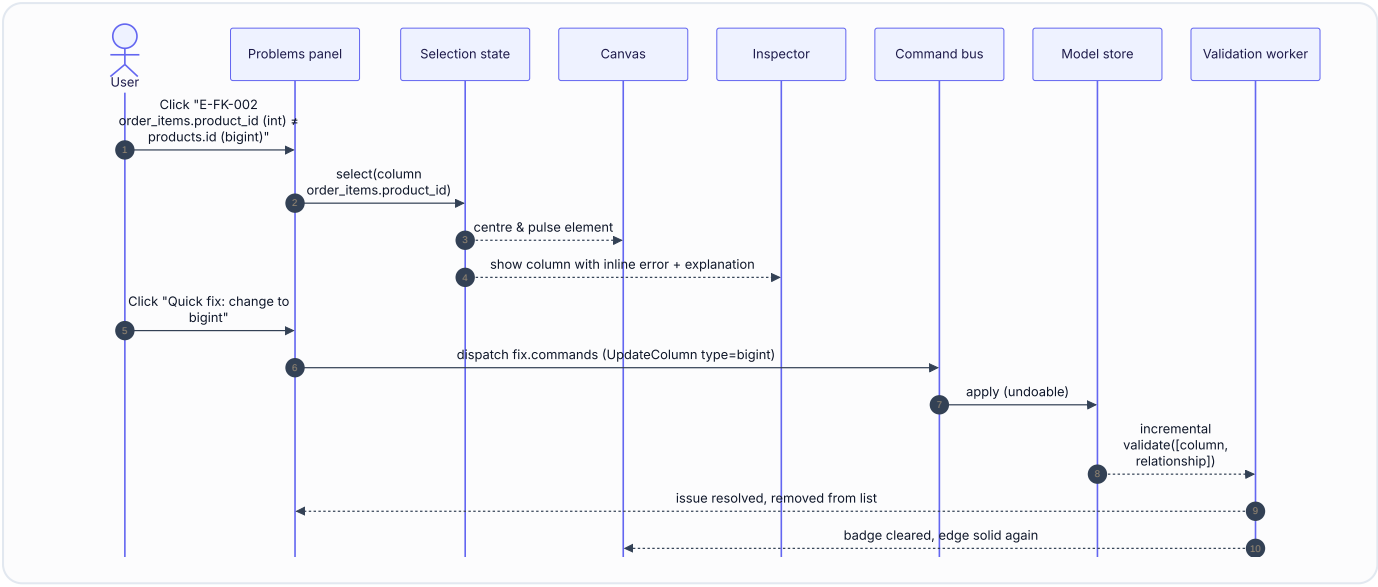


Dragging from a column onto an **empty** area of another table creates a new child column there, copying the parent's type and naming it `<parent>_<column>` (e.g. `customer_id`), then links it.

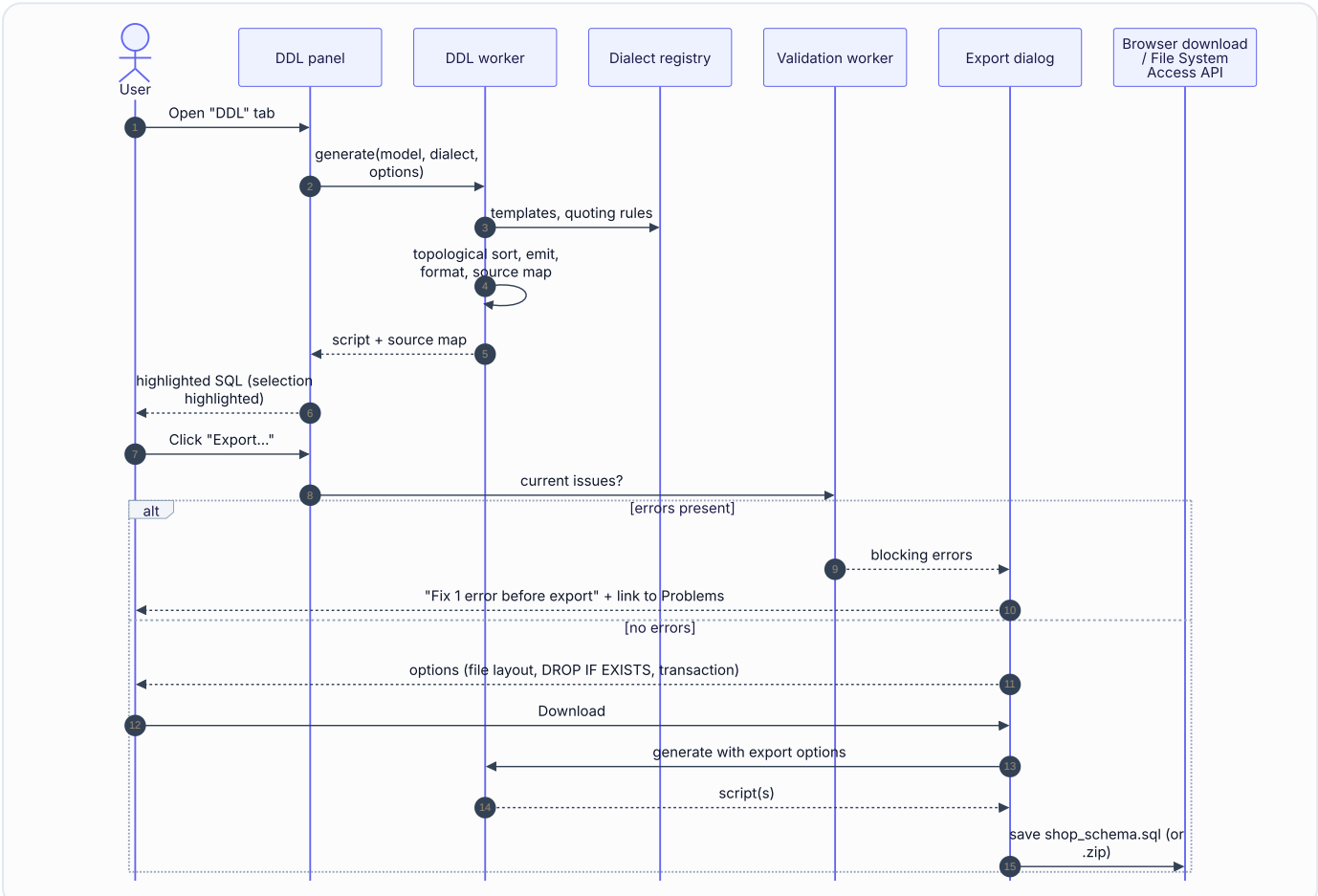
10.5 Select an engine and revalidate



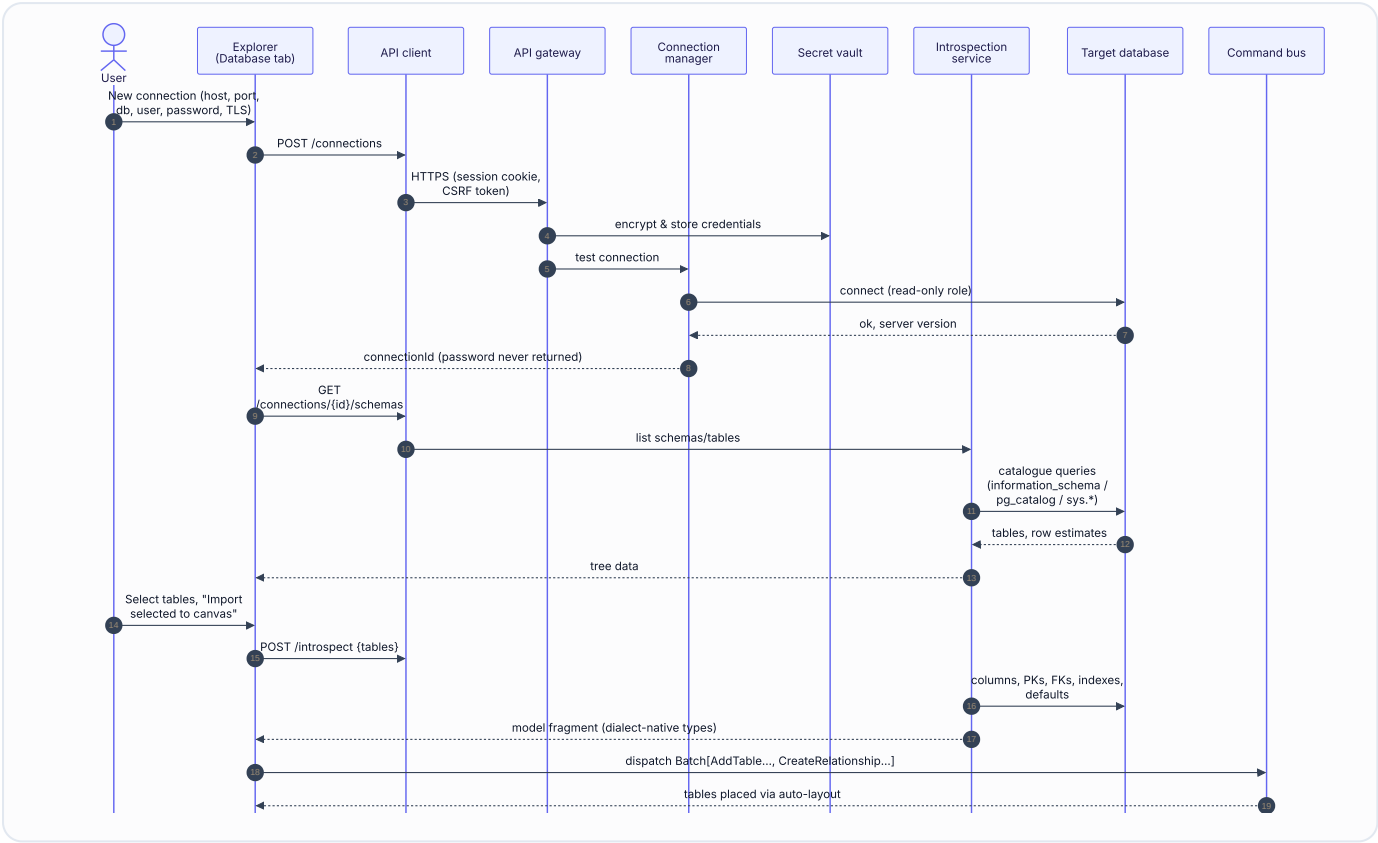
10.6 Navigate to a problem and apply a quick fix



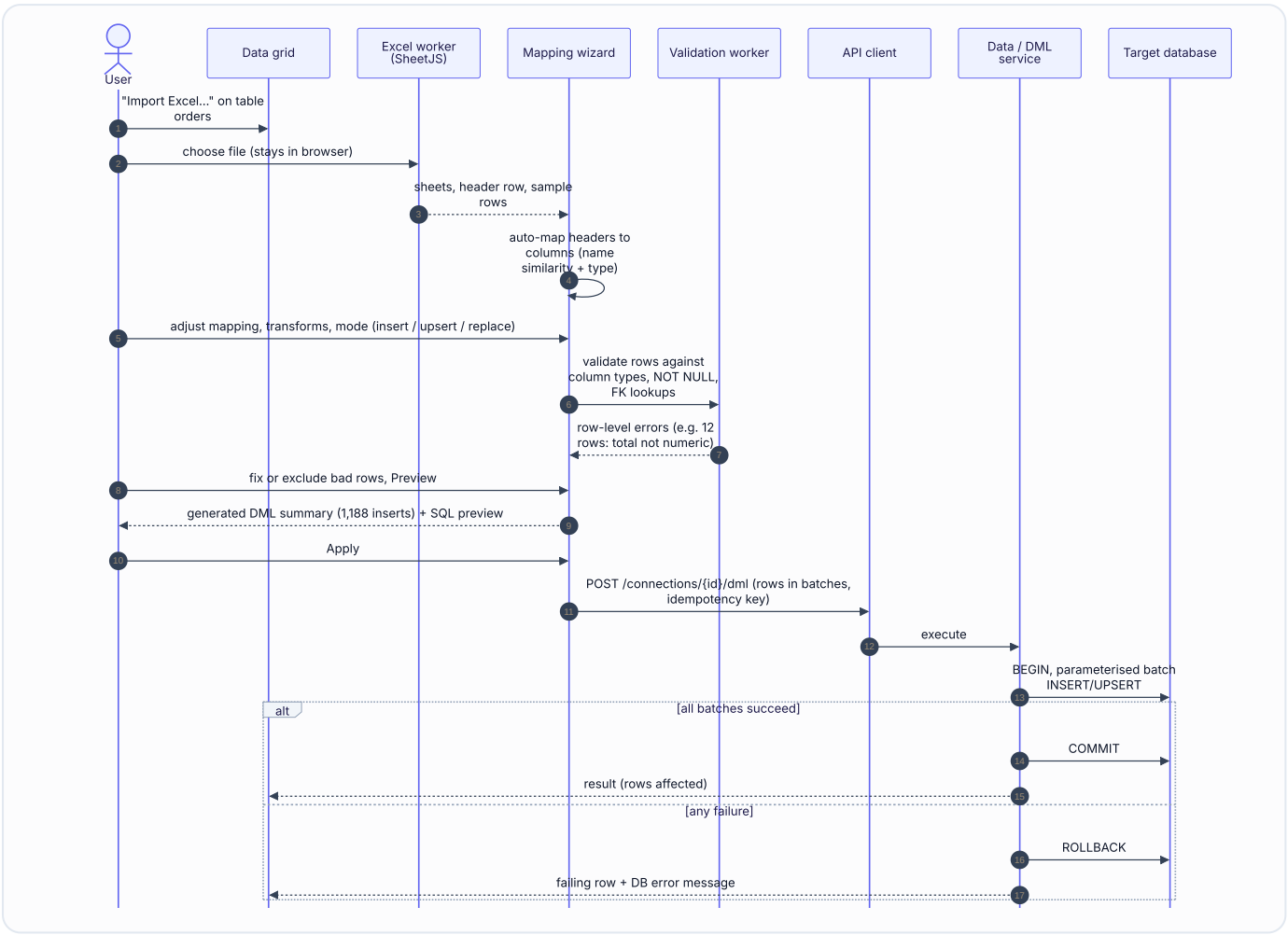
10.7 View and export DDL



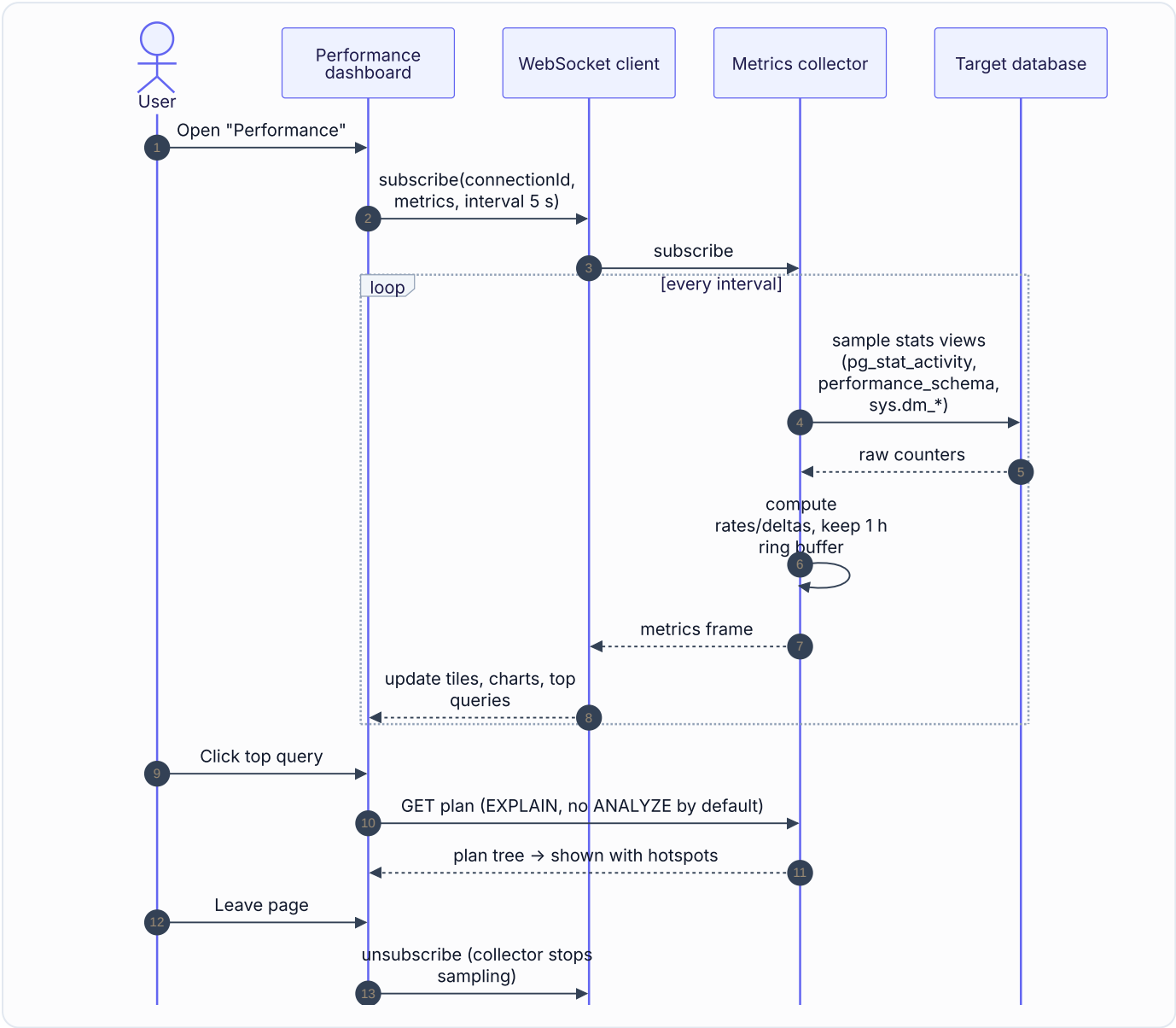
10.8 Connect to a database and reverse-engineer (Phase 6)



10.9 Excel import to DML (Phase 7)



10.10 Live performance metrics (Phase 8)



11. Backend and API (Phase 6+)

11.1 Deployment view

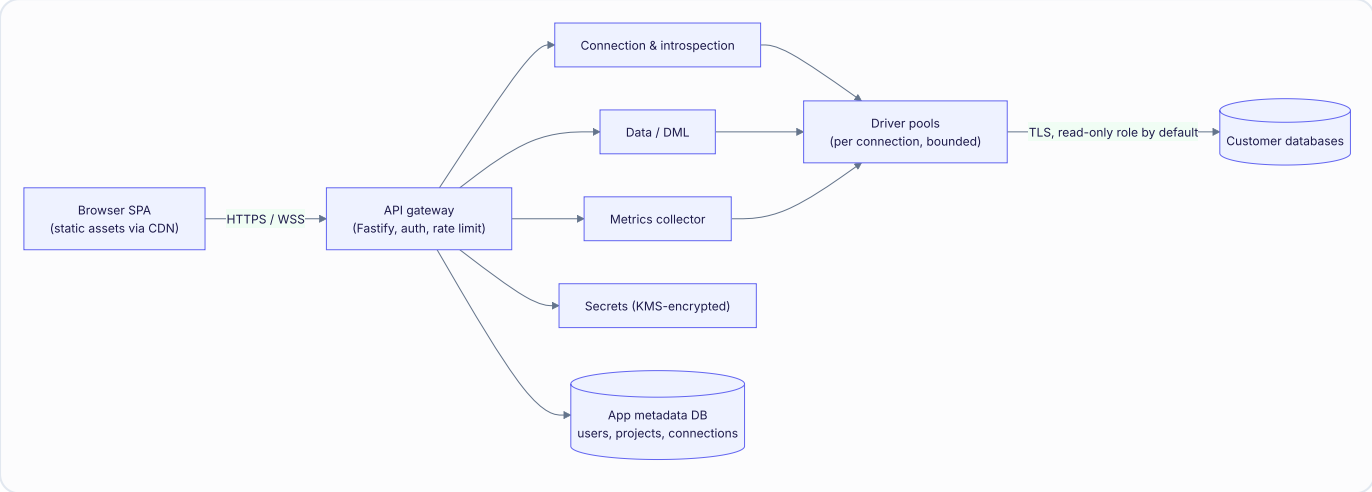


Figure 7 — Deployment view. The backend is stateless apart from connection pools, so it scales horizontally. It can be deployed on-premises next to the databases.

11.2 API surface (indicative)

Method	Path	Purpose
`POST`	`/api/connections`	Create connection (credentials stored encrypted)
`POST`	`/api/connections/{id}/test`	Test connectivity, return server version
`GET`	`/api/connections/{id}/catalog`	Schemas, tables, views with row estimates
`POST`	`/api/connections/{id}/introspect`	Return a model fragment for selected tables
`GET`	`/api/connections/{id}/tables/{t}/rows`	Paged, sorted, filtered rows (keyset pagination)
`POST`	`/api/connections/{id}/dml`	Execute a batch of parameterised DML in one transaction
`POST`	`/api/connections/{id}/ddl/apply`	Apply generated DDL (explicit opt-in, confirmation required)
`WS`	`/api/connections/{id}/metrics`	Stream metrics frames

11.3 Security

- **Credentials** are sent once over TLS, encrypted at rest with envelope encryption (KMS), and never returned to the client.
- **Least privilege.** Connections are read-only unless the user enables write mode for that connection. DML and DDL apply require write mode plus an explicit confirmation that shows the exact SQL.
- **No SQL injection.** All DML is built server-side from structured requests with bound parameters. Identifiers are validated against the introspected catalogue and quoted by the dialect.
- **Auditing.** Every write operation is logged with user, connection, statement hash and row counts.
- **Network.** Optional allow-list of target hosts. Statement timeouts and row limits are enforced server-side.

12. Quality and testing strategy

Level	What	Tooling
Unit	Commands and inverses (property test: apply then undo gives the original), each validation rule, type mappings, name generation	Vitest + fast-check
Golden files	DDL output per dialect for a corpus of models (cycles, composite keys, reserved words)	Vitest snapshots
Engine conformance	Generated DDL executed on real engines; introspecting it back must give an equivalent model (round-trip)	Testcontainers: PostgreSQL, MySQL, SQL Server, Oracle Free; SQLite in-process
Component	Inspector forms, Problems panel, type pickers	React Testing Library
End-to-end	The sequences in §10, on desktop and mobile viewports	Playwright (+ visual regression)
Performance	1,000-table model: render, drag, validate, generate budgets	Playwright traces, benchmark CI job
Accessibility	Keyboard-only flows, axe checks	axe-core, manual screen-reader passes

13. Risks and open questions

Risk / question	Impact	Mitigation / proposal
Engine rules differ across versions (e.g. Oracle `BOOLEAN` only in 23ai)	Wrong validation results	Dialects are versioned. Rules declare `minVersion` / `maxVersion`, backed by conformance tests
Canvas performance on very large schemas	Poor user experience	Viewport culling, simplified level-of-detail rendering when zoomed out, subject-area views
Reverse-engineering edge cases (computed columns, domains, custom types)	Incomplete import	Import unsupported objects as "opaque" columns with a warning, never drop them silently
Live DML against production databases	Data loss	Read-only default, transaction plus preview, confirmation, audit log, optional "non-production only" flag
Open: hosted SaaS, on-premises, or both?	Backend design, auth	Proposal: build stateless and container-first so both are possible; decide before Phase 6
Open: project storage — local files only or server-side projects with sharing?	Collaboration features	Proposal: local-first in Phases 1–5; server projects optional from Phase 6
Open: which engine versions are "tier 1" at launch?	Test matrix size	Proposal: PostgreSQL 16/17, MySQL 8.4, SQL Server 2022, Oracle 23ai, SQLite 3.4x